SECTION 07 · RECOMMENDER SYSTEMS

Recommender Systems from Implicit Feedback

Learn tastes from who watched what: popularity, item similarity, ALS, EASE and a transformer that predicts the next movie.

Implicit ALS | SASRec | EASE | Recall@10 | NDCG@10 | MovieLens 1M

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

Recommenders Everywhere

Because you watched
Toy Story

Customers also bought
this kettle

Up next
in your playlist

People you may know
on a network

The only data: who did what, and when.

No one labels a movie as "right for this user". The system learns it from behaviour.

Explicit vs Implicit Feedback

Explicit feedback

movie 1	★★★★★
movie 2	★★☆☆☆
movie 3	not rated
movie 4	★★★★☆
movie 5	not rated

rare: few people rate

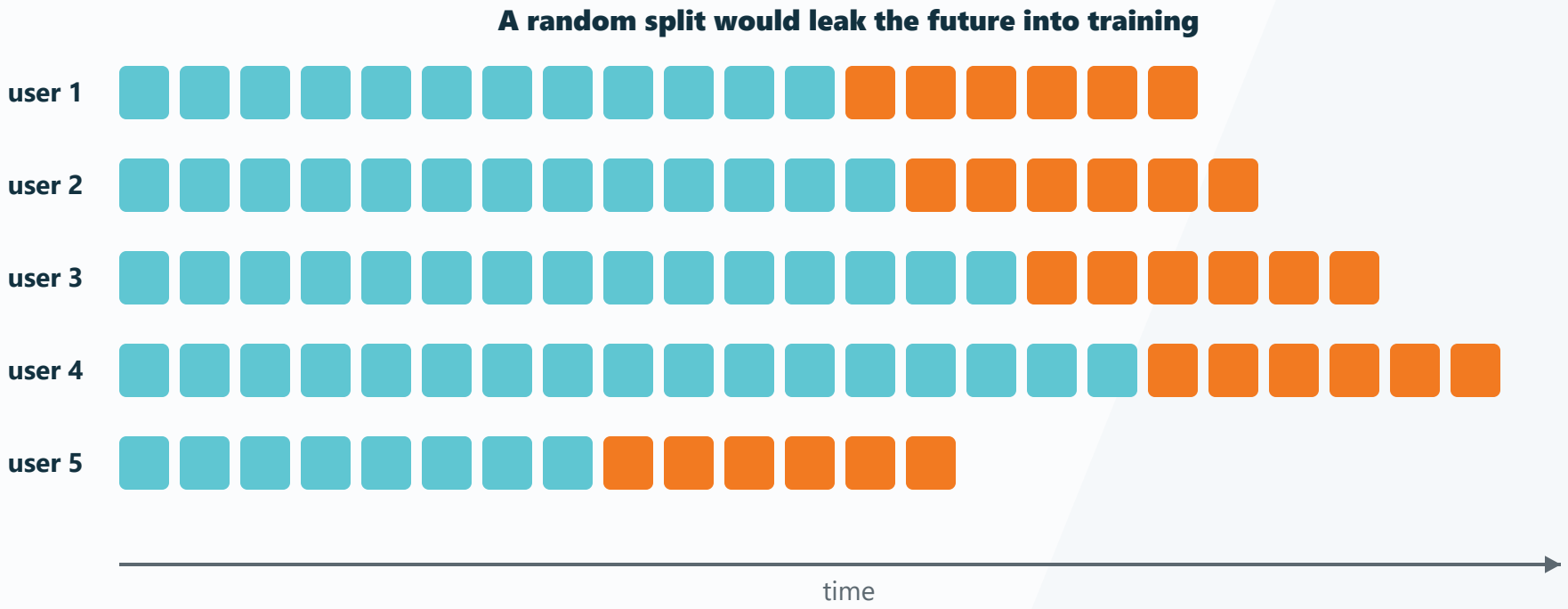
Implicit feedback

movie 1	watched
movie 2	watched
movie 3	?
movie 4	watched
movie 5	?

plentiful: every click, view, purchase

A missing entry is not a dislike: the user may never have seen the movie.

Holding Out Each User's Last Movies

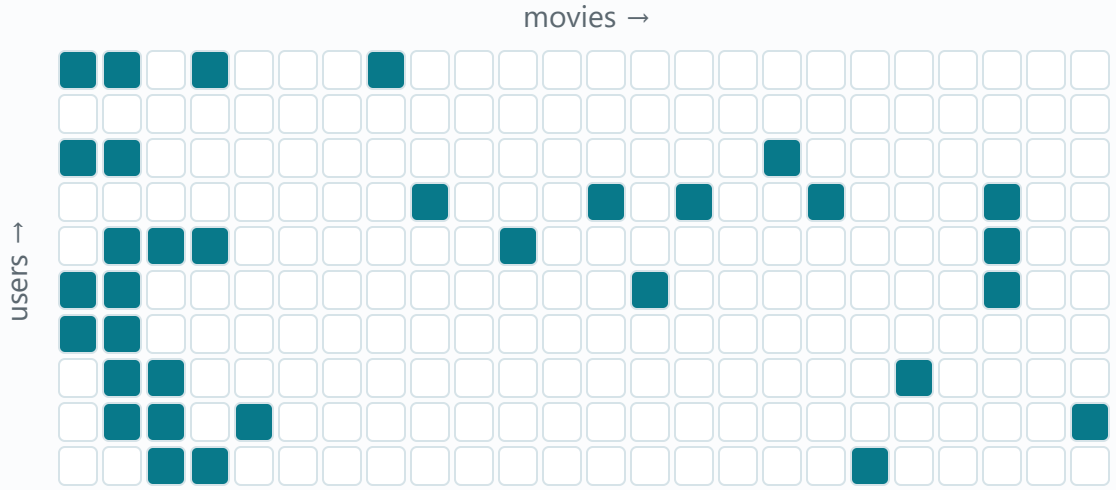


train
on the past

test
each user's last movies (10 in the notebook)

USERS BY MOVIES, ALMOST ALL EMPTY

The Watch Matrix



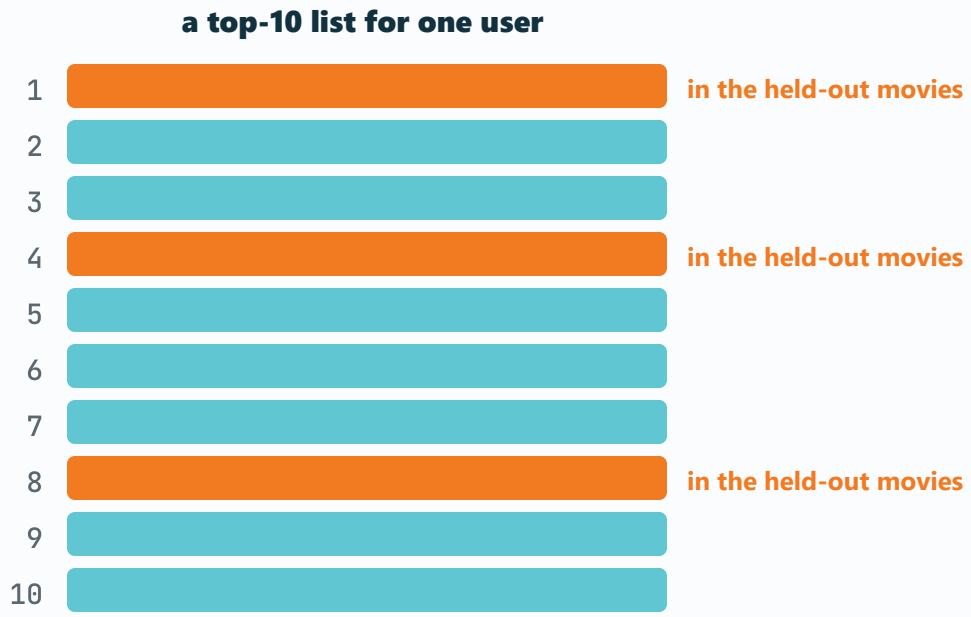
a few popular movies fill whole columns; most cells are empty

Rows: users, columns: movies
1 where the user watched the movie

Real matrices are mostly zeros, so they are stored sparse: only the filled cells.

scipy.sparse.csr_matrix
the input to every model in this section

Recall@10 and NDCG@10



Recall@10

held-out movies found in the top 10, out of 10

here: 3 of 10

NDCG@10

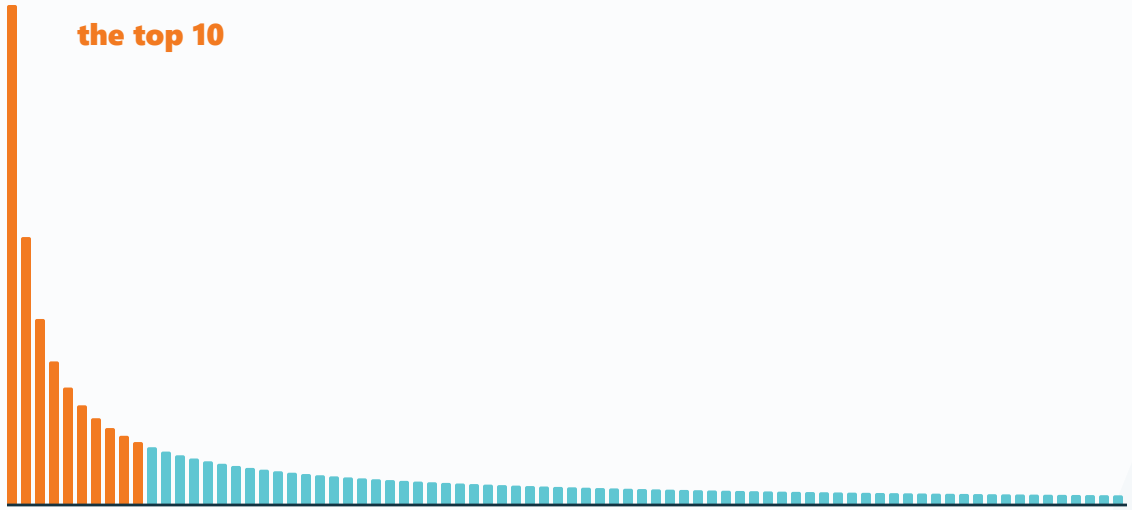
same hits, each worth less the lower it sits:

```
1 / log2(position + 1)
```

Averaged over all 6,040 users

both are 0 for random lists and 1 for perfect ones

The Popularity Baseline



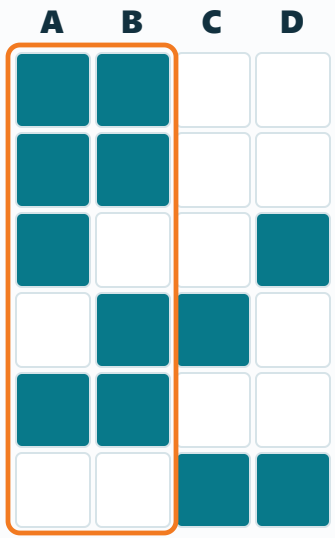
movies sorted by number of viewers: the long tail

Recommend the most watched movies the user has not seen

Strong, because popular movies really are watched by many

Same list for everyone
every model must beat it to be worth running

Item-to-Item Similarity



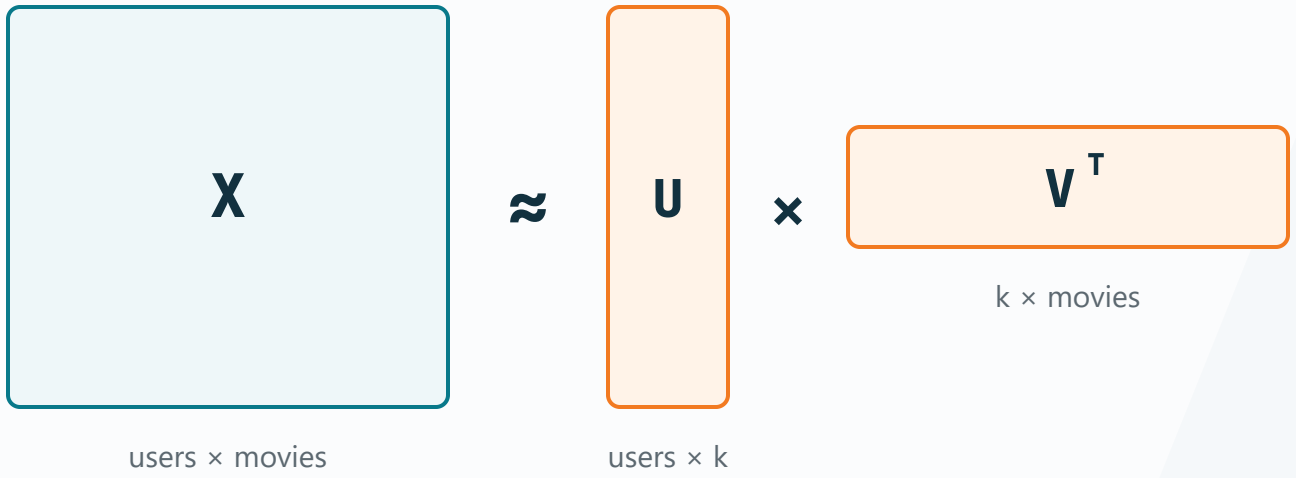
A and B: watched by the same people

$$\text{cosine}(A, B) = \frac{\text{people who watched both}}{\sqrt{(\text{watched A})} \cdot \sqrt{(\text{watched B})}}$$

Score for a user = sum of similarities to every movie they watched

"People who watched this also watched"
the classic recommender: simple, explainable, a strong baseline

Latent Factors as Taste Dimensions

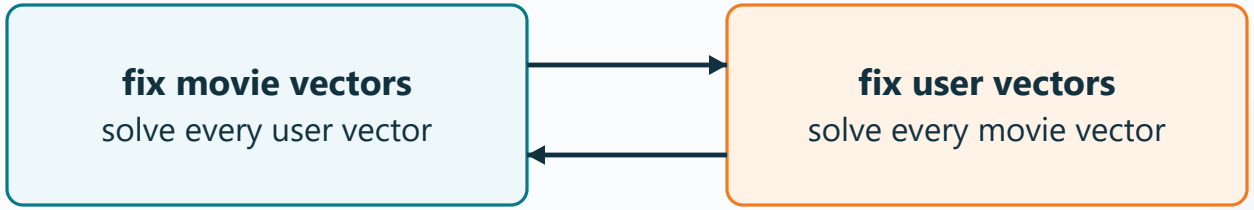


Each user and each movie get k numbers

a dimension might mean "animated", "older films" or "dark tone"; nobody names them, the data shapes them

Score = user · movie
high when their tastes line up

How Implicit ALS Learns



repeat: each step is a plain least-squares fit

fast on millions of users, runs on CPU or GPU

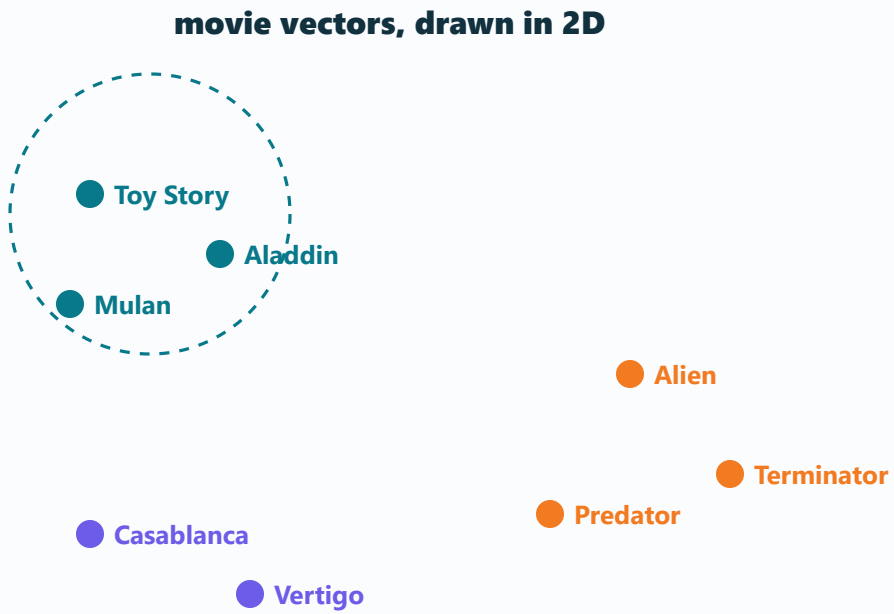
Implicit ALS treats every cell as data:

watched: target 1, high confidence

not watched: target 0, low confidence

Missing is not disliked
so empty cells pull only gently

"More Like This" From Item Vectors

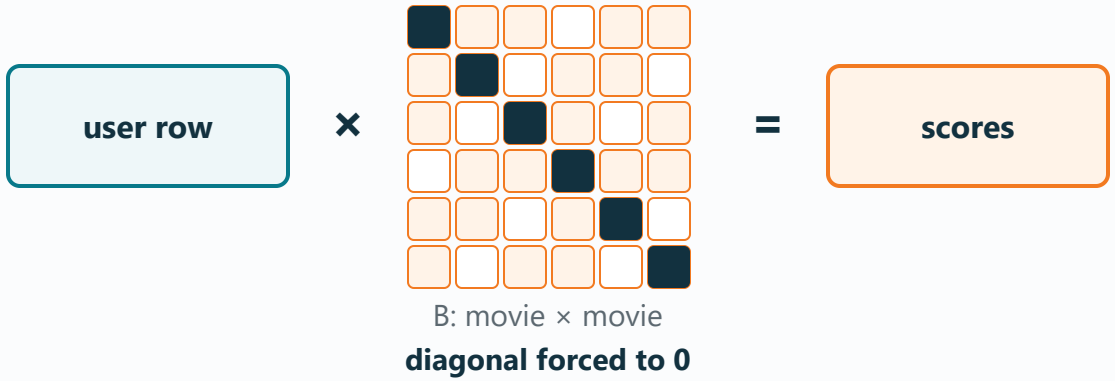


Similar movies sit near each other

"More like this" = the nearest movie vectors by cosine similarity

Learned only from who watched what
no genres, plots or actors were given

EASE: A Linear Autoencoder Over Items



An autoencoder with no hidden layer

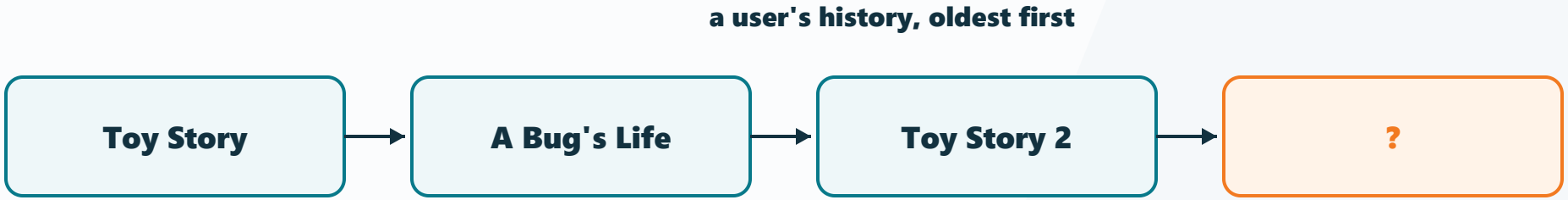
predict each movie from all the others;
a zero diagonal stops a movie from simply copying itself

One matrix inverse, no training loop

$B = -P / \text{diag}(P)$, $P = (X^T X + \lambda I)^{-1}$

λ (lam) is the only setting

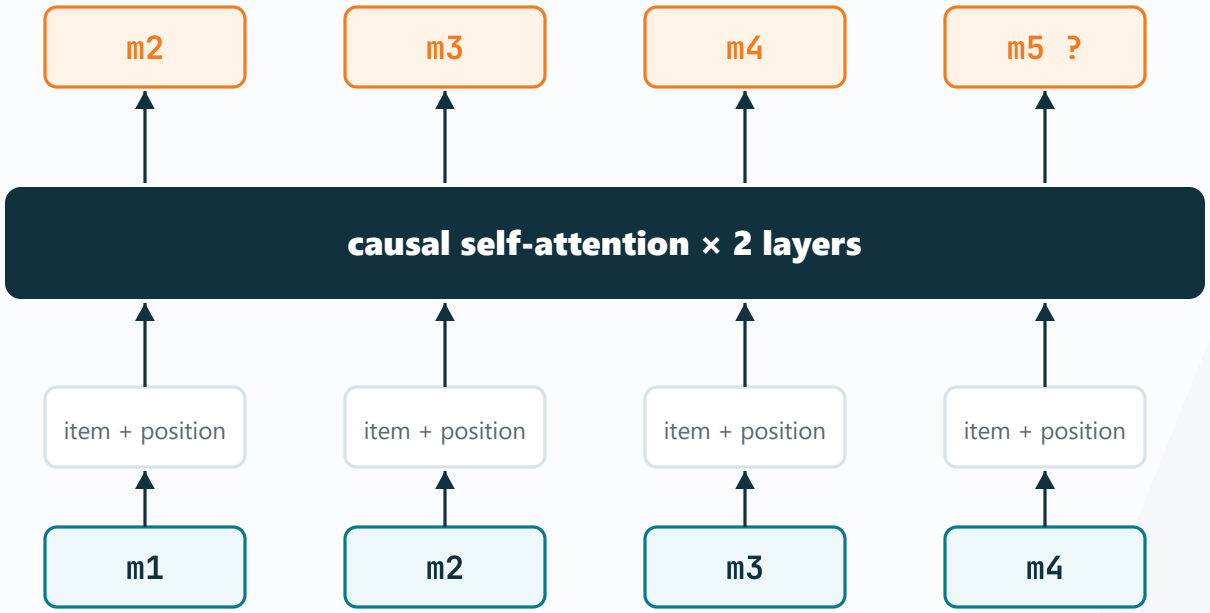
The Order of Watching Matters



ALS and EASE see a bag of movies. The order of watching is thrown away.

What someone watched last says the most about what they watch next.

How SASRec Predicts the Next Movie



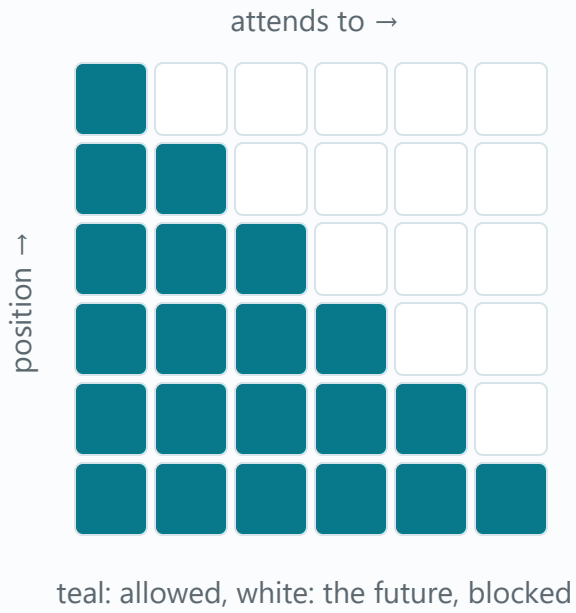
targets: the next movie at every step

each step attends to earlier ones only

embeddings learned from scratch

the history, oldest first

Next-Item Prediction as Self-Supervision



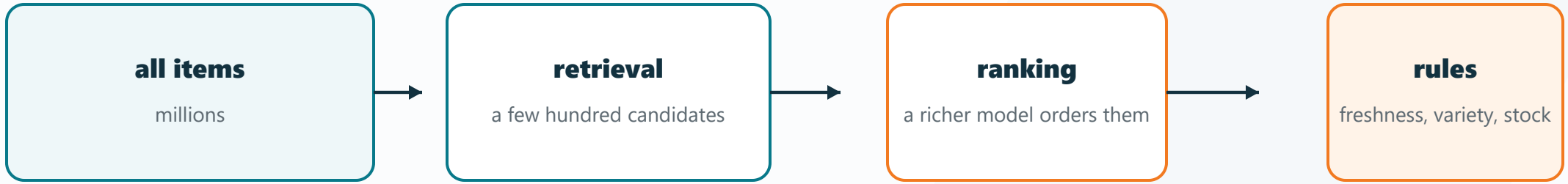
teal: allowed, white: the future, blocked

One sequence, many training examples

position 1 predicts movie 2, position 2 predicts movie 3, ...
the mask stops any position from peeking ahead

Self-supervised
the labels are the user's own next movies, so no one writes them;
the same recipe as next-word prediction in language models

Retrieval Then Ranking



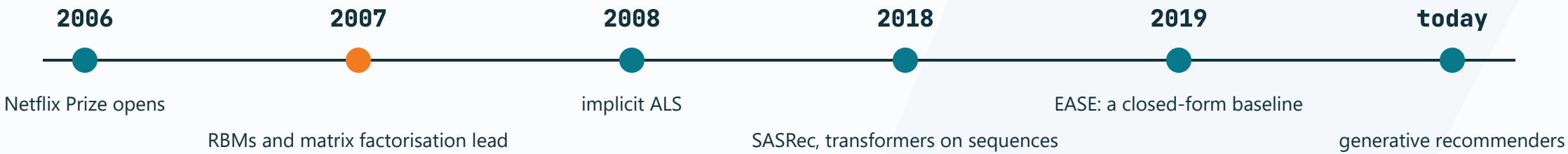
the catalogue

ALS, SASRec, embeddings with a nearest-neighbour index

user, item and context features

This section builds the retrieval stage: the part learned without labels.

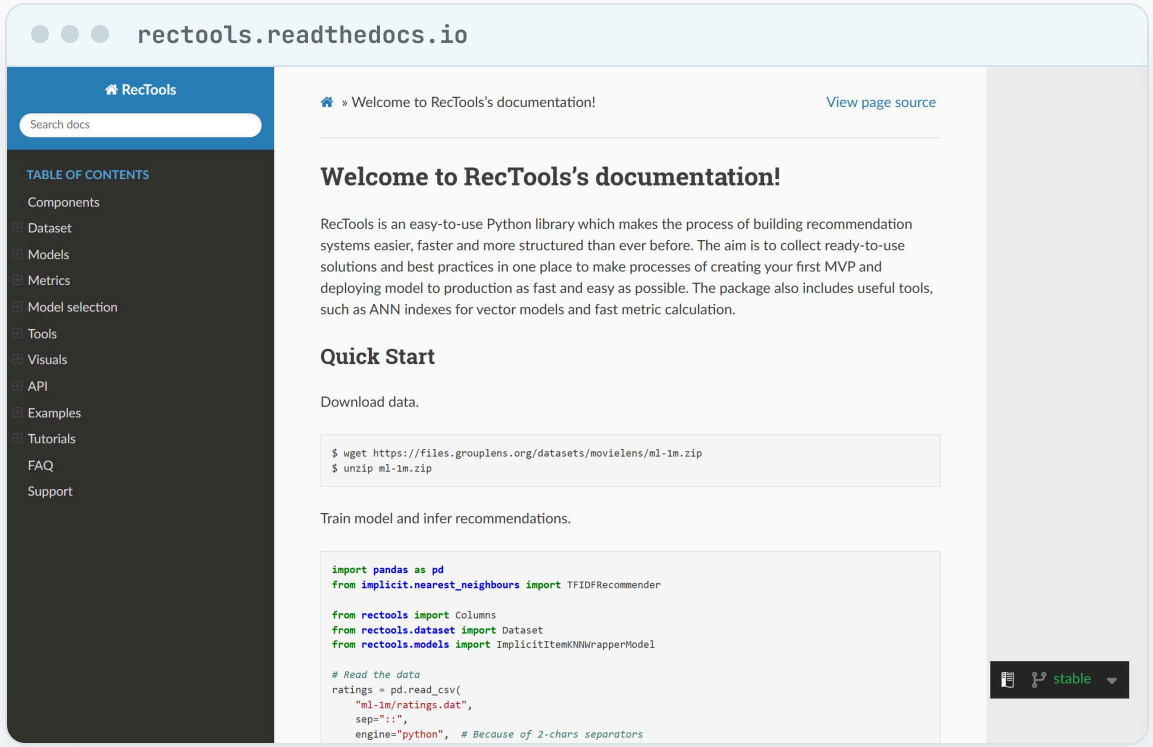
From RBMs to Sequence Transformers



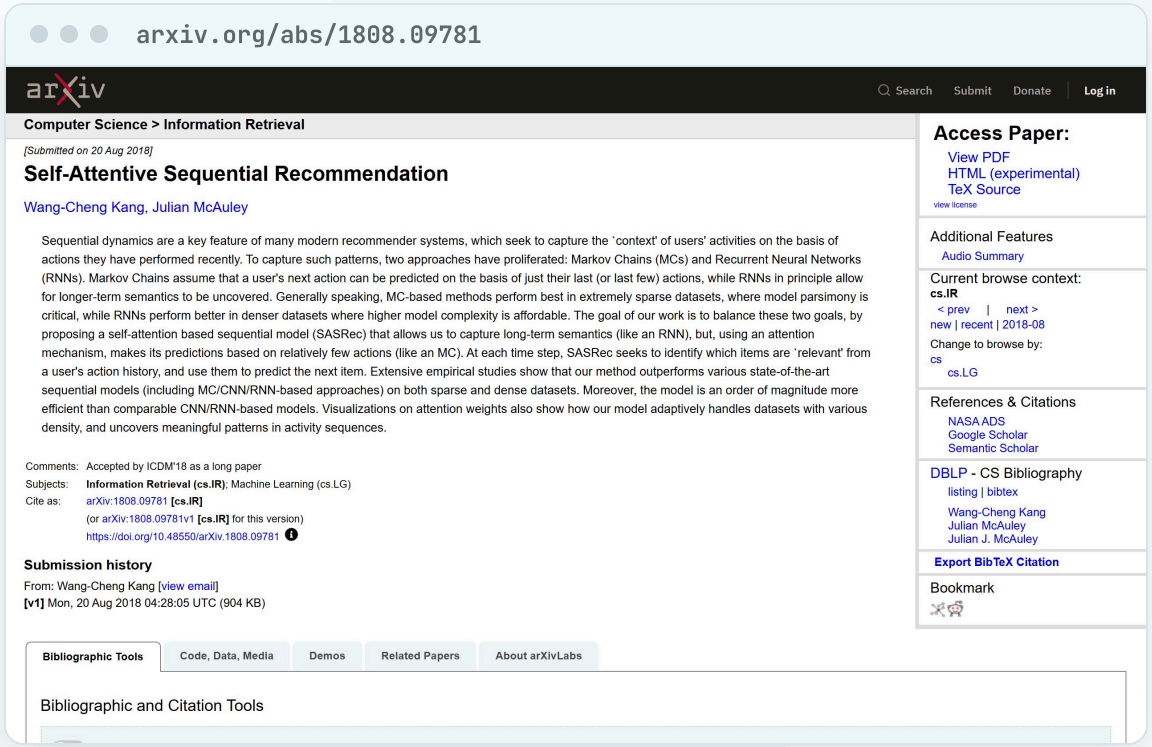
The book's chapter uses restricted Boltzmann machines

they shaped the field, but factor models, EASE and sequence transformers replaced them

RecTools Docs and the SASRec Paper



RecTools: one fit and recommend for popularity, item similarity, ALS, EASE and SASRec



The SASRec paper: self-attentive sequential recommendation

RecTools wraps the implicit library for item similarity and ALS, and builds SASRec on PyTorch Lightning.

Choosing a Recommender

METHOD	USES ORDER	TRAINING	STRENGTH	USE IT FOR
Popularity	no	a count	hard to beat for new users	baseline, cold start
Item similarity	no	one matrix product	easy to explain	"also watched" lists
Implicit ALS	no	seconds, scales far	user and item vectors	retrieval, similar items
EASE	no	one inverse	very strong for its size	catalogues up to tens of thousands
SASRec	yes	a GPU helps	knows what came last	"up next", session-based feeds

SECTION 07 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie